PORTALINK

SYSTEM PROJECT DOCUMENTATION

Online Portfolio Template Generator

A concise guide to the Portalink system, its database, deployment, features, and current verification status.

Prepared for project submission

Published website
https://portfolio-production-07cf.up.railway.app/

01 / PROJECT OVERVIEW

Project title & description

1. Project Title

Portalink - Online Portfolio Template Generator

2. Project Description

Portalink helps people create a personal portfolio website without building one from scratch. A user creates an account, enters profile and career information, chooses one of three designs, and shares a public link to the finished portfolio.

The system is intended for students, job seekers, and creative professionals who want to present their work, experience, and contact information in one place. The owner can return to the account dashboard to edit or delete portfolios.

Portfolio details

Name, email, phone, location, profile image, introduction, education, skills, projects, experience, and links.

Three visual styles

Simple, Modern, and Creative templates offer different layouts and visual treatments.

Account workspace

Sign in to create, view, edit, preview, and delete portfolios you own.

Public sharing

Each portfolio has a public page. Contact links use the owner's email and saved links.

System flow

Home → Create account / Sign in → Enter information → Save
↓ **Choose one of 3 styles → Preview → Share public link → Edit or delete when needed**

Public link: https://portfolio-production-07cf.up.railway.app/
Individual public portfolios use a unique /p/{slug} address.

02 / TECHNICAL SUMMARY

Technologies, database & hosting

3. Technologies Used

Category	Technology	Purpose
Language	PHP 8.3+	Runs server-side application code.
Backend framework	Laravel 13	Routes, authentication, validation, database access, and page rendering.
Page templates	Blade, HTML, CSS	Builds the account pages, dashboard, forms, and portfolio themes.
Frontend build	Vite, Tailwind CSS 4, Node.js 22, npm	Builds and serves frontend assets.
Database	PostgreSQL on Supabase (deployment configuration)	Stores accounts, portfolio details, and application records.
Data layer	Laravel Eloquent and migrations	Maps PHP models to database tables and creates their schema.
Photo storage	Supabase Storage, S3-compatible API	Stores uploaded profile images; the portfolio row stores the image path or URL.
Hosting	Railway, Docker, Apache	Builds and runs the deployed Laravel web application.
Health check	Laravel /up endpoint	Lets Railway check that the web process is responding.
Tests	PHPUnit / Laravel test runner	Automated feature and unit tests are included in the source project.

4. Database Platform

The project migrations define a relational schema. The Railway deployment guide configures Laravel to connect to **PostgreSQL hosted by Supabase** through the Supabase session pooler, with SSL required. Database credentials are stored as private Railway environment variables and are not included here.

Laravel reads connection settings from environment variables. The repository's local default is SQLite; the documented production setting is PostgreSQL (`DB_CONNECTION=pgsql`). Portfolio records can be created, retrieved, updated, and deleted through Laravel's Eloquent models and controller actions. Profile-photo files are stored separately in Supabase Storage when that disk is selected.

> **Verification boundary:** The public home page and Railway `/up` health endpoint responded successfully during this documentation work. That health check does not prove database reads/writes; a full production registration and portfolio CRUD test is still needed.

03 / DATABASE DESIGN

6. Database Structure

The database schema is based on the project's Laravel migrations. The central portfolio data is stored in one `portfolios` row; education, skills, projects, work experience, and social links are JSON columns, not separate tables.

```
Relationship overview

users (id, primary key) 1 ── 0..* portfolios (user_id, nullable foreign key)
portfolios.user_id → users.id; deleting a user sets the portfolio owner field to NULL.
sessions.user_id is nullable and indexed; the migration does not declare it as a foreign key.
password_reset_tokens, cache, jobs, and related framework tables have no portfolio relationship.
```

users

Column	Key / rule	Purpose
`id`	Primary key	Account identifier.
`name`	Required string	Account holder's name.
`email`	Required, unique	Sign-in email address.
`email_verified_at`	Nullable timestamp	Email verification time, if used.
`password`	Required string	Hashed account password.
`remember_token`	Nullable	Remember-me authentication token.
`created_at, updated_at`	Timestamps	Record creation and last update.

portfolios

Column	Key / rule	Purpose
`id`	Primary key	Portfolio identifier.
`user_id`	Nullable foreign key; indexed; null on user deletion	Owner account; references users.id.
`slug`	Required, unique, up to 24 characters	Public portfolio URL identifier.
`full_name, email`	Required strings	Public profile name and contact email.
`phone, address`	Nullable strings	Contact number and location.
`about`	Nullable text	Profile introduction.
`profile_picture`	Nullable string	Image URL or stored object path.
`education, skills, projects, experience, links`	Nullable JSON	Lists of portfolio entries saved in the portfolio record.
`template`	String; default simple	Selected design: simple, modern, or creative.
`is_complete`	Boolean; default false	Draft/completion flag.
`created_at, updated_at`	Timestamps	Record creation and last update.

04 / DATABASE DETAILS

Supporting tables & operations

Other tables created by migrations

Table	Important fields / constraints	Purpose
password_reset_tokens	`email` primary key, `token`, nullable `created_at`	Laravel password-reset support. The current routes do not include a password-reset page.
sessions	`id` primary key, nullable indexed `user_id`, IP address, user agent, payload, indexed activity time	Database session schema. Railway setup documents file-based sessions.
cache and **cache_locks**	Primary key `key`; value/owner and indexed expiration fields	Laravel cache and lock schema. Railway setup documents file-based cache.
jobs	Auto-increment ID, indexed queue, payload, attempts, reservation/availability/creation times	Database queue records; Railway setup selects the database queue.
job_batches	String primary-key ID, job counters, failed IDs, options, status timestamps	Queue batch tracking.
failed_jobs	Auto-increment ID, unique UUID, connection, queue, payload, exception, failure time	Records failed queued jobs.
migrations	Laravel-managed migration log	Tracks schema changes already applied.

How portfolio data is handled

- On save, the controller validates the submitted fields and converts each multi-line list into a JSON array.
- On retrieval, the signed-in dashboard lists portfolios belonging to that account; the public page is found by its unique slug.
- On edit and delete, ownership is checked before the portfolio record is changed or removed.
- Uploaded images use the configured storage disk; external image URLs can also be saved. Image files are separate from the relational database.

> **Data model note:** There are no separate education, skill, project, experience, or social-link tables in the migrations. Those lists are JSON fields in `portfolios`.

5. Hosting / Deployment Platform

The application is deployed on **Railway** using the project's Dockerfile. The image builds frontend assets with Node.js, installs PHP dependencies, and runs Laravel with Apache. On startup, the container applies Laravel migrations and listens on Railway's assigned port. Railway is configured to check `/up`.

The documented production data services are Supabase PostgreSQL and Supabase Storage. The live website URL is:

https://portfolio-production-07cf.up.railway.app/

The home page and health endpoint were reachable during this check. Database credentials and storage keys are intentionally omitted.

05 / ACTUAL INTERFACE

7. Screenshots of the System

These images were captured from the published Portalink website. They show the pages that can be opened without an account.

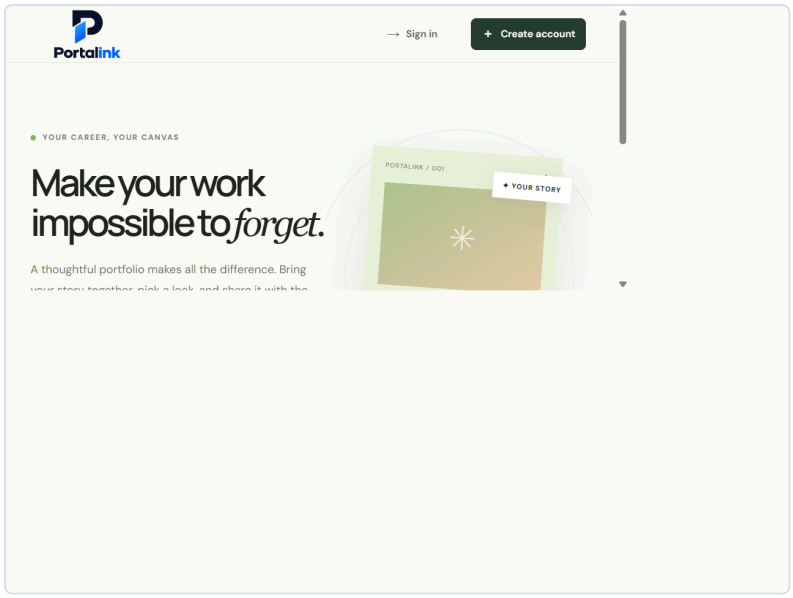


Figure 1. Public home page w ith the Portalink introduction and account entry buttons.

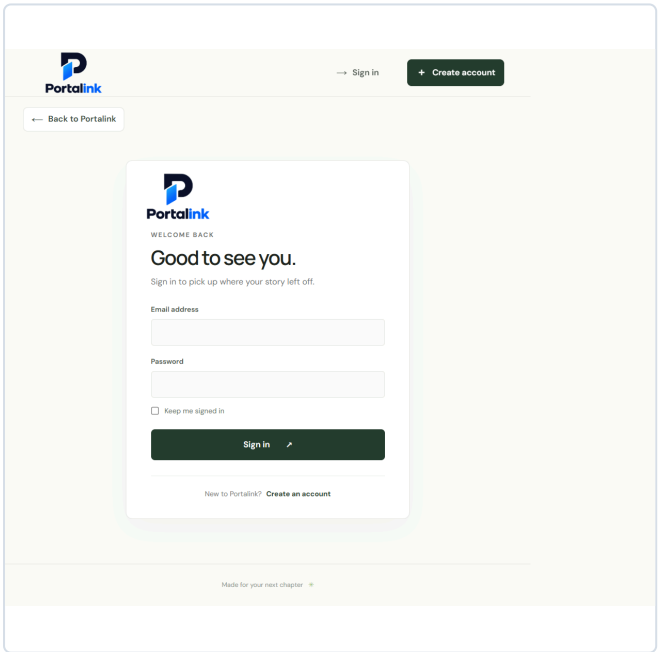


Figure 2. Sign-in page. Authentication w as not submitted.

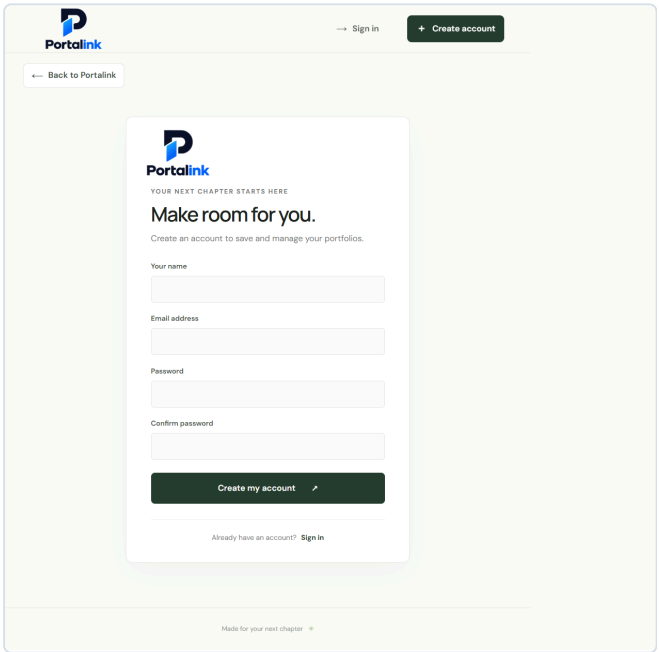


Figure 3. Account creation form. No account w as created for this documentation.

> The remaining screens require a signed-in account. Their placeholders are identified on the next page; they are not represented as real screenshots.

06 / MAIN SYSTEM FEATURES

8. Important Features

SCREENSHOT PLACEHOLDER

My portfolios dashboard
Capture after signing in.

Figure 4. Dashboard. Lists the user's portfolios and offers create, preview, edit, and delete actions.

SCREENSHOT PLACEHOLDER

Portfolio information form
Capture a filled or blank form.

Figure 5. Portfolio form. Collects personal details, education, skills, projects, experience, links, and an optional profile image.

SCREENSHOT PLACEHOLDER

Three template choices
Capture all three in one image.

Figure 6. Template selection. The available choices are Simple, Modern, and Creative. The source includes different preview treatments for each.

SCREENSHOT PLACEHOLDER

Portfolio preview and owner controls
Capture a selected theme.

Figure 7. Preview. Shows the selected theme, a back-to-dashboard control, copy-link action, style change, and edit details.

How the main features work

Feature	User action	Expected result
Account access	Register or sign in.	Access to the private portfolio workspace.
Create / edit	Complete or update the form and save.	Validated portfolio data is stored for the signed-in owner.
Choose a design	Select Simple, Modern, or Creative.	The chosen design is saved and used by the preview/public page.
Preview / share	Preview the result or copy its public link.	Owner preview and public page display the portfolio using its slug.
Delete	Choose Delete and confirm.	The portfolio is removed and the user returns to the dashboard.

> **Image options:** The form accepts a profile-image URL or JPEG, PNG, GIF, or WebP upload up to 5 MB. Upload behavior depends on the configured storage service.

07 / REQUIREMENTS CHECK

Teacher's requirement checklist

Status reflects source-code review and the checks completed for this report. "Implemented but not verified" means the relevant code or page exists, but the full production interaction was not tested here.

Requirement	Status	Evidence / remaining check
Project title and description	**Implemented and verified**	Portalink branding and public homepage reviewed.
Home page with description and account/create entry	**Implemented and verified**	Live homepage screenshot captured; page opened successfully.
Registration and sign-in pages	**Implemented but not verified**	Live forms opened and captured; form submission was not performed.
Portfolio information fields required by the assignment	**Implemented but not verified**	Fields and validation exist in the form/controller; create-and-save was not tested against production.
Online/cloud database	**Implemented but not verified**	Railway configuration targets Supabase PostgreSQL; verify a real portfolio row in Supabase.
Add, save, retrieve, edit, and delete portfolio data	**Implemented but not verified**	Routes and controller actions exist; complete each action using a test account.
Exactly three noticeably different templates	**Implemented but not verified**	Simple, Modern, Creative are defined; capture the live selector and one preview per theme.
Preview and public portfolio display	**Implemented but not verified**	Owner preview and public slug route exist; open a real saved portfolio link.
Dashboard for viewing/managing portfolios	**Implemented but not verified**	Authenticated dashboard exists; capture it and test its actions.
Data remains available after refresh	**Implemented but not verified**	Requires a production save, refresh, and database check.
Public website and working URL	**Implemented and verified**	Website and `/up` health endpoint responded during this check.
Simple, readable, navigable interface	**Partially implemented**	Public/auth pages reviewed; authenticated pages and mobile layout still need a manual review.
System screenshots, including all three templates	**Partially implemented**	Three public/auth screenshots are included; account-only pages are labeled placeholders.
Project documentation and safe database details	**Implemented and verified**	This report describes schema and deployment without private keys or passwords.

> **Test environment note:** A local test rerun could not start because the available CLI PHP is 8.2.12; the project requires PHP 8.3 or newer. This report does not claim a new local test-suite pass.

08 / WRAP-UP

10. Workflow & conclusion

System workflow

Open Portalink → Register / Sign in → Enter portfolio information
↓ **Save to the online database → Choose a template → Preview → Share the public URL**

Return to dashboard → Edit or delete the portfolio

11. Conclusion

Portalink is a web application for creating and sharing personal portfolios. Its source includes account access, portfolio management, three portfolio styles, a public portfolio page, optional profile-image storage, and a Laravel/PostgreSQL deployment configuration for Railway and Supabase.

The public site and health endpoint were reachable during this review. Full production account, database, upload, and portfolio-management behavior still needs to be demonstrated with a test account before submission.

Actions before submission

1. Open **https://portfolio-production-07cf.up.railway.app/** and test account registration and sign-in.
2. Create a portfolio, save it, refresh, and confirm the saved row in Supabase's `portfolios` table.
3. Try editing and deleting a test portfolio; verify each of the three styles and open the public share link in a signed-out browser.
4. Replace Figures 4-7 with actual screenshots from the signed-in app, including all three template previews and the public portfolio page.
5. Submit the source code and name Supabase as the online database platform. Do not submit database passwords, app keys, or storage secrets.

Prepared from the Portalink source files, Laravel migrations, deployment guide, and live public pages. The teacher's instructions were used as the requirements checklist; the other group's sample was used only as a presentation reference.